

1.流水线概述

核心：并行运行，提高吞吐率

阶段：IF取指令-ID指令译码和读取寄存器-EX执行或计算-MEM从数据存储器读取数据-WB写回寄存器

设计：每一个 stage 只需要 1 个时钟周期，因此时钟周期必须能满足最慢的阶段执行；我们可以将寄存器的写操作发生在时钟周期的前半段，该操作发生在时钟周期的后半段

考点：三种Processor 对比

	周期时间	总时间	说明
单周期	$CT_{seq} = \sum_{k=1}^N T_k$	$Time_{seq} = 1 \times CT_{seq} = \sum_{k=1}^N T_k$	所有阶段在一个周期内完成，周期由最慢阶段决定
多周期	$CT_{multi} = \max(T_k)$	$Time_{multi} = cycle \times \frac{CT_{multi}}{N} = Average(CPI) \times CT_{multi}$	每周期执行一个阶段；CPI 因指令不同需取平均
流水线	$CT_{pipeline} = \max(T_k) + T_d$	$Time_{pipeline} = cycle \times \frac{CT_{pipeline}}{(1+N-1) \times CPI} \times T_d$	流水线周期等于最慢阶段 + 寄存器开销；初始有 N-1 填充周期

补充：Average CPI=Σcpi/CPIxf
CPI_i: 第 i 类指令所需周期数
f_i: 第 i 类指令在所有指令中的比例 (frequency)
N: 指令类型数量

考点：流水线的优势——加速比

Speedup_{pipeline} = Time_{seq} / Time_{pipeline} ≈ (ΣN_kT_k) / (I × T_k) = N

理想情况：

(1) 所有流水线阶段时间相同：ΣN_kT_k = N × T_k

(2) 无流水线开销：T_d = 0

(3) 理想情况下，远大于流水线阶段数 N (可忽略 N-1)

结论：在理想情况下，流水线处理器的速度提升约等于流水线的阶段数 (N)

理想情况解释了流水线如何损失性能：stage 耗时各不相同；流水线寄存器本身带来延迟，各指令 hazard (冒险)

补充：时钟周期时间=1/时钟周期频率

2.流水线冒险 (Hazard)：

结构冒险：

原因：硬件不支持多指令令在同一时钟周期执行 (这就是为什么 MIPS 流水线采取 Harvard architecture，分离指令存储器和数据存储器)

数据冒险：

原因：一个流水级必须等待另一个流水级完成，从而导致流水线造成暂停的情况

解决：前推 (forwarding) /旁路 (bypassing)，具体做法是不需要等待指令执行结束，直接从内部缓冲器而非程序可见的寄存器或存储器中提取出数据 (只有当目标级在时间上晚于前推级旁路才会有效)

注意：而推不能级快 load-use data hazard，流水级不得不阻塞一个流水级的时间 (stall/bubble)。解决方案：(1) 硬件上检测和阻塞 (2) 软件上重新安排代码顺序

控制冒险：

原因：决策依赖于一条指令的结果，而其他指令正在执行中

解决：阻塞 (stall) 取出分支指令后立即阻塞流水线直到确定分支结果并确定下一条真正要执行的指令地址 (恶语但慢)

解决：预测 (predict) 有以下两个版本

(1) 预测一直不发生：预测正确时流水线会全速执行；只有当分支发生时才会阻塞；

(2) 预测一直发生：与上述相反；但是 (1) 效率会更高，因为 (2) 需要提前跳转地址 (一般要等到 EX 阶段才知道)，而 (1) 可以直接跳转执行；

(3) 延迟决定：MIPS 编译器会在延迟分支指令的后面放一条不受该分支影响的指令，具体操作是把分支指令要执行的指令在分支跳过的指令移动到延迟指令，或者跳转后的第一条指令如果不会受到分支的影响那也可以放延迟逻辑，实在没有就用 nop 指令；

(4) 预测分支：预测一些分支发生一些不发生。比如保存每次分支的历史记录，利用最近的历史行为来进行预测

Lec10 Cache

1. 动机

Memory Wall (存储墙): CPU 的速度增长远快于 DRAM 内存的速度。CPU 可能需要等待 50 个时钟周期才能获取数据

局部性原理 (Principle of Locality): 程序在短时间内倾向于访问一小部分内存空间

时间局部性 (Temporal Locality): 如果一个数据被访问，它很快会被再次访问 (例如：循环变量)

空间局部性 (Spatial Locality): 如果一个数据被访问，它附近的数据也会很快被访问 (例如：数组遍历)

目标：制造一种“像 SRAM 一样快，像 DRAM 一样大/便宜”的存储系统

2. 核心计算公式

Lec13 Scheduling

1. Process 类型

CPU 密集型大部分时间在 CPU 上运算 (如图形处理) 可以后台处理

I/O 密集型大部分时间在等待 I/O 设备 (如读写磁盘、网络、键盘输入) 需要交互和更快的响应

2. 多任务环境类型

类型	特点	备注
批处理 (Batch)	实际上不是多任务，一次只运行一个进程直到结束	
协作式 (Co-operative)	运行中的进程不能被调度器强制挂起，必须主动放弃 CPU (Voluntary)	依赖 yield() 函
抢占式 (Pre-emptive)	运行中的进程可以被调度器强制挂起 (Suspended forcefully)	现代 OS 主流方式

3. 考点：Scheduling Policies

(1) 通用策略：固定优先级 (Fixed Priority): 任务被分配优先级 (通常 0 为最高)，总是运行就绪队列中优先级最高的任务

(2) 批处理策略：FCFS (先来先服务): 简单的队列，公平但可能导致由于等待前面长作业而产生的饥饿

SJF (短作业优先): 按总 CPU 时间排序，减少平均等待时间，但长作业可能饥饿

(3) 协作式策略：Round Robin (Round Voluntary Scheduling): 进程主动调用 yield() 来切换

(4) 抢占式策略：Round Robin with Timer (RR): 每个进程分配一个固定时间片 (Time Slot)，时间到则被抢占

SRT (最短剩余时间): SJF 的抢占式版本，按剩余所需时间排序

两类时间的计算

• 响应时间 response: $T_{response} = T_{first_run} + T_{turnaround}$

• 等待时间 waiting: 每次在就绪队列等待的时间

• 周转时间 turnaround: $T_{turnaround} = T_{completion} - T_{arrival}$

注意：计算每个进程的等待时间时不仅算等待前一个进程的时间，还要算上之前进程的等待时间！

为什么高优先级任务仍然可能被最低优先级中断打断，以及这如何影响 I/O？

Hardware interrupts and software task priorities operate on different levels of control

Interrupts are handled by the CPU's interrupt controller, not the OS scheduler. When an interrupt occurs, it preempts whatever the CPU was doing, even if that's the highest priority task.

The CPU has no concept of "task priority", it just fetches and executes instructions. The task priority is only relevant to the task scheduler.

Interrupt lines are checked at the end of each instruction execution cycle, so they are always serviced regardless of their priority level

4. Linux 调度

A. 进程分类与优先级

实时进程 (Real-time): 优先级 0-99.

RT-FIFO: 除非有更高优先级 RT 进程，否则不可被抢占

RR-RR: 有时片轮转；时间片轮转调度

普通非实时进程 (Non-real-time): 优先级 100-139

Nice 值: 用户可调整优先级。范围-20 到 19 (负数优先级更高，仅 root 可用负数)

B. 调度数据结构

双队列机制 (Two Sets of Queues): 维护 280 个队列，分为两组，每组 140 个 (对应 140 个优先级)

• Active Set (活跃组): 存放还有剩余时间片的进程

• Expired Set (过期组): 存放时间片用完的进程

算法核心: 调度器总是从 Active Set 中选最高优先级的运行。当 Active Set 为空时，交换 Active 和 Expired 的指针 (O(1)复杂度)，Expired 变为新的 Active C. 动态计算与启发式

时间片 (Time Quantum): 优先级越高，时间片越长 (例如优先级 100 有 800ms，优先级 139 只有 5ms)

优先级计算公式: $SPriority = base + f(nice) + g(cpu_usage)$ 。如果消耗过多 CPU 时间，优先级会衰减

I/O Boost (I/O 奖励): 为了保证交互响应，Linux 会“奖励”那些经常等待 I/O 的进程 (如键盘输入)

奖励方式：增加时间片或提升优先级 (例如提升-5 的优先级)

Lec14 IPC

Core Goal: 处理多进程间的协作与竞争，确保数据一致性。

Lec12 Process Management

1. 进程启动

Bootloader: 系统启动时运行的第一个程序，负责加载 OS 内核 (在 Wintel 系统中位于存储盘的 MBR)

General Flow:

Power on -> Execute BIOS/UEFI from Read Only Memory (ROM) -> Perform Power On Self Test (POST) -> Load bootloader -> Bootloader locates and loads kernel into RAM -> Kernel starts master process (init/systemd) -> Initialize device drivers, kernel modules, background processes -> Launch user GUI

Kernel: OS 的核心部分：几乎持续运行。

System Programs: 用于程序访问、OS 配置和系统维护。

2. The Onion Model

Hardware-Microcode-Machine Instructions-Device Drivers-Kernel-Application

3. 考点：内核结构对比

特性	Monolithic Kernel	Microkernel
定义	OS 的所有主要部分都在“内核空间”运行 (不是硬件)	只有“核心”部分在内核空间，其他在“用户空间”
内核空间内包含	调度器、内存管理、文件系统、设备驱动、IPC 等。	仅包含：调度器、进程管理、内存管理。
用户空间内包含	应用程序	文件系统、设备驱动 (如 USB 驱动) 作为系统服务运行
可扩展性	支持运行时加载/卸载模块 (如 Linux 的 modprobe)	模块化程度更高
典型例	Linux, MS Windows	MacOS

4. 系统调用 (System Calls): 用户程序与操作系统 API 之间的接口。

双重模式 (Dual Mode):

User Mode (用户模式): 运行普通应用程序。

Kernel Mode (内核模式): 运行内核代码，执行特权操作。

切换方式

类型	来源	典型例子	处理后的行为
Interrupt	外部硬件	鼠标点击、时钟中断	返回下一条指令
Trap (系统调用)	软件指令	read(), getpid()	返回下一条指令
Fault (语义异常)	指令语义错误	缺页异常、除零错误	重新执行当前指令 (如缺页) 或终止进程

1. 进程：内存中正在运行的实体。包括程序代码以及上下文

与程序的关系：一个程序可以产生多个 CPU

2. 中断 (Interrupts)

触发：当硬件设备 (如 IO 控制器) 需要 CPU 注意时，通过 IRQ (Interrupt Request) 线发送信号

流程：CPU 暂停当前程序的顺序执行-控制权跳转到 ISR (中断服务程序)-ISR 执行完毕后，CPU 返回原程序断点继续执行

3. 并发与上下文切换

• 并发：通过时间片轮转 (Time-slicing) 实现“并发”的假象

• 上下文切换：

关键步骤：

(1) 保存：将当前进程的 PC 程序计数器)、寄存器、状态字 (SREG) 保存到内存

(2) 恢复：从内存加载新进程的 PC 和寄存器信息

SREG (状态寄存器): 记录溢出、零标志位、中断使能等信息，对于条件跳转 (if 语句) 至关重要

4. 进程状态转换

状态	描述	转换触发
Running (运行)	正在 CPU 上执行	-
Ready (就绪)	准备好运行，等待调度器分配 CPU	时间片用完或被抢占 (Running -> Ready)
Blocked (阻塞)	等待外部事件 (如 I/O 输入)	需要数据 (Running -> Blocked)

注：阻塞和就绪，进程会进入 Ready 状态，而不是直接进入 Running

5. 关键数据结构：PCB

PCB (Process Control Block): 操作系统为每个进程维护的数据结构

内容：包含 PID (进程 ID)、栈指针、打开的文件列表、CPU 使用情况等

存储：所有 PCB 存储在系统的进程表 (Process Table) 中

6. 进程的创建与终止

• 创建：使用 fork() 系统调用

• 父进程 fork() 返回子进程的 PID (n > 0)

• 子进程 fork() 返回 0，父进程会复制父进程的代码、堆栈等资源，但 fork 这一行返回后将继续执行

所有 Unix 进程最终都是 init 进程的子进程

(2) 终止与僵尸进程：

进程终止时，资源 (文件等) 被释放，但 PCB 会保留

正常清理：父进程调用 wait() 等待子进程运行结束获取结果，系统释放子进程

Lec11 Intro to OS

1. 概念：两个或多个进程读写共享数据，最终结果取决于进程运行的精确时序 根源：操作不是原子的

2. 临界区 (Critical Sections) 访问共享资源的那段代码

3. 考点：实现机制对比

特性	TSL (Test & Set Lock)	Semaphores (信号量)
类型	硬件	软件
等待方式	忙等待 (Busy Waiting) while(lock) 循环，浪费 CPU。	阻塞 (Sleep/Block) 主动放弃 CPU，进入等待队列。
状态	进程保持 RUNNING 态	进程进入 BLOCKED 态
缺陷	可能导致忙等待死循环 (高优先级死循环，低优先级无释放锁)。	使用顺序不当会导致死锁。

4. 考点：信号量 (Semaphores)

UP: DOWN (P): 如果值 > 0，减 1 继续；如果值 = 0，睡觉 (Block)

UP: V: 值加 1；如果有睡着的进程，叫醒 (Wakeup) 其中一个

生产者-消费者问题

变量: mutex=1 (保护缓冲区的锁), empty=N (空位数), full=0 (产品数)。

Producer (生产者):

produce_item(); // 1. 造数据 (临界区外)

down(&mutex); // 2. 先申请空位 (无位则睡) <- 【死锁关键点：必须在 mutex 前】

down(&full); // 3. 后申请锁

insert_item(); // 4. 放数据 (临界区)

up(&mutex); // 5. 释放锁

up(&full); // 6. 增加产品数 (可能唤醒消费者)

// Consumer (消费者)

down(&full); // 1. 先申请产品 (无货则睡)

down(&mutex); // 2. 后申请锁

remove_item(); // 3. 取数据 (临界区)

up(&mutex); // 4. 释放锁

up(&empty); // 5. 增加空位数 (可能唤醒生产者)

consume_item(); // 6. 吃数据 (临界区外)

注: mutex = 1 没上锁; mutex = 0 锁了

5. 常见错误

• 信号量死锁 (Deadlock)

场景：生产者把 down(&mutex) 放在了 down(&empty) 前面

UP: V: 缓冲满了 -> 生产者拿了锁 -> 生产者等空位(sleep) -> 消费者想消费但没锁(block) -> 死锁。

• 优先级反转 (Priority Inversion)

角色: Task A (高), Task B (中), Task C (低)。

过程: C 拿了锁; A 想要锁，被阻塞; B 抢占了 C 的 CPU (B > C)。

结果: C 无法运行释放锁，导致 A 一直等。中优先级 B 间接阻塞了高优先级 A

Lec15 Mem management

1. 物理地址存储

(1) 字节序

大端 (big endian): 高位字节 (MSB) 存放在低地址 (如 Sparc)

小端 (little endian): 低位字节 (LSB) 存放在低地址 (如 x86)

• 字节序是偶 16 进制数组成，千万别搞反过来才！

(2) 对齐

按 word 对齐。否则读取效率极低 (需要多次读取并拼接)，或者直接触发硬件故障 (Bus Error)。

Padding 问题: 编译器会在结构体成员之间插入无用的字节，以确保后续成员字节对齐。Char 占 1 字节

2. 内存管理机制

逻辑地址：进程代码中“看到”的地址

物理地址：实际放入内存硬件的地址

逻辑 -> 物理：根据 segment 序号找到 base，加上偏移量

(1) 多进程保护

Base Register (基址寄存器): 存放进程的起始物理地址

Limit Register (界限寄存器): 存放内存的长度

保护机制：如果访问地址 < Base 或 > Base + Limit，触发 Segmentation Fault

(2) 分区：将内存划分为固定区域给进程使用

碎片 (Fragmentation) 的类型：

Opcode

ALUOp

Instruction Operation

Funct field

ALU action

ALU control

lw

00

load word

XXXXXX

add

0010

sw

00

store word

XXXXXX

add

0010

beq

01

branch equal

XXXXXX

subtract

0110

R-type

10

add

10 0000

add

0010

R-type

10

subtract

10 0010

subtract

0110

R-type

10

AND

10 0100

AND

0000

R-type

10

OR

10 0101

OR

0001

R-type

10

set on less than

10 1010

set on less than

0111

